

Module 21: Class Design in Depth

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

Module 20 decided which classes a program needs; this module makes each class correct and hard to misuse. Students learn why member initializer lists are initialisation rather than assignment, give every member a safe default, remove duplicated validation with delegating constructors, stop silent conversions with `explicit`, and make `const` correctness a habit. The conceptual centre is the class invariant - and the realisation that a getter and setter for every member protects nothing. Static members and out-of-class definitions round the module off and prepare for Module 22's header files.

Suggested Duration: 90 minutes

Learning Objectives

By the end of this module, students will be able to:

- Initialise every member in a member initializer list, and explain why `const` and reference members require it.
- Use default member initializers so no member is ever left uninitialised.
- Remove duplicated validation with a delegating constructor, and mark single-argument constructors `explicit`.
- Apply `const` correctness to member functions and parameters.
- State a class invariant and design public methods that preserve it, instead of blind setters.
- Use static data members and static member functions, and define methods outside the class with `ClassName::`.

Prerequisites

- Module 20 (OOP Principles & Object Modelling).
- Part 1 Module 8 (constructors, `this`) and Module 7 (pass by `const` reference).

Materials & Instructor Prep

- 21-class-design/examples/ 01 to 06 preloaded; 03_`const`_correctness.cpp and 04_invariants.cpp are the key demos.
- Be ready to uncomment the deliberately broken lines in examples 02 and 03 to show the compiler errors live.

Key Vocabulary

Term	Definition
Member initializer list	The list after a constructor's colon that constructs members directly with their starting values, before the constructor body runs.
Default member initializer	A starting value written where a member is declared, used by every constructor that doesn't override it.
Delegating constructor	A constructor whose initializer list calls another constructor of the same class, so shared work is written once.
explicit	A keyword on a single-argument constructor (or conversion operator) that forbids silent, automatic conversions.
const member function	A method marked const after its parameter list, promising not to modify the object; callable on const objects and through const references.
Class invariant	A rule that is true for every object of a class at all times between method calls, established by the constructor and preserved by every method.
static member	A data member or function that belongs to the class itself, shared by all objects rather than copied into each.

Lesson Outline & Instructor Talking Points

1. Initialise, don't assign (15 min)

- Run examples/01_initializer_lists.cpp. Ask students to try rewriting Patient's constructor with assignments in the body - the const and reference members refuse to compile.
- Show the -Wreorder warning by deliberately listing initializers in the wrong order. Explain that members are initialised in declaration order, whatever the list says.

2. Delegating and explicit constructors (15 min)

- Walk through examples/02_delegating_and_explicit.cpp: validation lives in one constructor, and the delegating one reuses it.
- Uncomment printResistor(330.0) to show explicit at work. Ask: did the caller mean 330 ohms, 330 kilo-ohms, or the wrong variable?

Instructor note: Students often see explicit as fussy. Frame it as making the caller say what they mean - the same idea as a function name being descriptive.

3. const correctness (15 min)

- Run examples/03_const_correctness.cpp. Remove const from getBalance() and show printStatement() no longer compiles. Make the point: forgetting const breaks other people's code.
- Uncomment the broken() method to show the compiler enforcing the promise.

4. Invariants: behaviour, not setters (20 min)

- Run examples/04_invariants.cpp. LeakyAccount happily holds -5000. Ask the class who is to blame - the class, because it offered setBalance().
- Introduce 'tell, don't ask'. Have students list the operations a BankAccount's users genuinely need, and design them with the invariant as a comment above the class.

Instructor note: *This is the conceptual heart of the module. Expect pushback ('but I need to set the balance when loading from a file'). Use it: loading is a constructor's job, which can validate.*

5. static members and out-of-class definitions (15 min)

- Run examples/05_static_members.cpp. Show the linker error if the definition of nextId is removed - preview of Module 22's linker errors.
- Run examples/06_out_of_class_definitions.cpp and point out the class body now reads like a table of contents. Note const must be repeated on the definition.

Common Student Misconceptions

Misconception	How to Address It
"An initializer list is just a shorter way to assign."	Members are constructed before the body runs. The list constructs them with the right value once; assignment in the body constructs then overwrites - and is impossible for const and reference members.
"Making data private and adding getters and setters is encapsulation."	A setter that accepts any value makes the data public in practice. Encapsulation means offering operations that keep the invariant true.
"const on a method is optional decoration."	Without it, the method can't be called through a const reference - so every function that takes a const& of your class breaks.
"A static member is a global variable."	It is shared like a global, but scoped to the class and subject to access control; it should model something that genuinely belongs to the class, such as an ID counter.

Exercises & Module Project

Exercise: 21-class-design/exercises/exercise1.cpp - a ClockTime class with an invariant, a delegating explicit constructor, const getters, wrap-around addMinutes(), a static counter, and out-of-class definitions. Project (three tracks): each track redesigns its core Module 20 class (Generic - Student and Club; EE - Appliance with a static tariff; Biomedical - Patient with generated folder numbers), annotating every Module 21 technique.

Facilitation guidance: Look for a written invariant comment on each class and check that no public method can break it - especially any remaining setters. Check const on every read-only method, and that static data members are defined outside the class. The requirement to label each technique with a comment makes marking quick.

Discussion & Comprehension Check Questions

1. Why can't a const member be set in the constructor body?

Guidance: By the time the body runs, the member has already been constructed and is const, so it can't be assigned. Only the initializer list constructs it with a value.

2. What goes wrong if `getBalance()` isn't marked const?

Guidance: It can't be called on a const `BankAccount` or through a const `BankAccount&`, so any function that receives the account read-only can't read its balance.

3. Give an example of an invariant for a `ClassPeriod` or `Thermostat` class, and a method that could break it.

Guidance: For example 'the target temperature is always between 16 and 30 C'; a `setTarget()` that doesn't check the range would break it.

Assessment Ideas

- Code review: give students a class with public setters for everything and ask them to rewrite its interface around operations and an invariant.
- Spot the bug: show an initializer list in the wrong order where one member depends on another, and ask what value it really gets.

Answer Key Notes

See `21-class-design/exercises/solutions/exercise1_solution.cpp`.

Wrap-Up & Bridge to Next Module

Bridge forward: "Our classes now declare their methods in the class body and define them outside it. Next module puts those two halves in different files - headers and source files - and introduces namespaces and CMake."

Instructor Tips

- If time is short, protect invariants (step 4) and const correctness (step 3); static members can be covered quickly from the examples.
- Keep returning to the question 'can any code outside this class put it into a nonsense state?'